

A Rede de Segurança

Onde você para de perguntar ao console se o código está certo

NÍVEL	MÉDIO	menos modelagem que o 01, mais rigor
STACK	Java 21 + JUnit 6 (um único <code>.jar</code> , sem build tool)	
SOLUÇÃO EM	<code>java/04-colecoes-streams/acervo-da-quarentena/test/</code>	
TEMPO ESTIMADO	3 a 5 horas, em duas sentadas (Ato I e Ato II)	
PRÉ-REQUISITO	desafio 01 aprovado — este desafio trabalha em cima daquele código	
CONCEITOS NOVOS	<code>@Test</code> , <code>assertEquals</code> com delta, <code>assertThrows</code> , <code>@BeforeEach</code> , <code>@DisplayName</code> , <code>@ParameterizedTest</code> , teste de mutação, refatoração sob rede	

1. O contexto

O acervo está catalogado. As patrulhas continuam trazendo caixa de coisa velha, e agora alguém da comunidade — provavelmente a Maria, que não confia em nada que ela mesma não tenha conferido — faz a pergunta que você vinha adiando: *"como você sabe que os números estão certos?"*

Na primeira entrega do desafio 01, a sua resposta teria sido "o programa imprimiu". E o programa imprimiu `1,4 min` ao lado de `esperado 2,8` e concluiu, na linha seguinte, que estava tudo bem. Foram quatro rodadas até aquela última linha ficar verde. Você não errou por não saber Java — o polimorfismo estava certo desde a primeira entrega. Você errou porque não tinha como saber que estava errado.

"Você conferiu?" — "Rodou sem erro."
"Não foi isso que eu perguntei."

O seu `check()` foi um bom começo — e não estou sendo gentil, foi mesmo: você identificou que `assert` vinha desligado e construiu a alternativa certa sozinho. Agora vamos trocar aquele começo por uma ferramenta que o mercado inteiro usa, e que responde coisas que o seu `check()` não responde: quantos testes passaram, quais falharam, por quê, e o que exatamente era esperado contra o que veio.

2. O que você vai construir

Este desafio tem **dois atos**, e a ordem é obrigatória. Fazer o Ato II antes do Ato I é justamente o erro que este desafio existe para curar.

ATO	O QUE É	POR QUE NESTA ORDEM
I — Montar a rede	Escrever a suíte de testes JUnit do acervo, sem tocar em nenhuma linha de <code>src/</code> .	Rede primeiro. Você precisa saber que o código funciona <i>hoje</i> para provar que continua funcionando amanhã.
II — Usar a rede	Fazer a faxina que ficou pendente do desafio 01, com a suíte verde a cada passo.	É aqui que a rede paga o investimento. Você vai mexer em código que funciona — sem medo, porque tem quem avise.

2.1 Onde os arquivos moram

```

java/04-colecoes-streams/acervo-da-quarentena/
├── src/                                # seu código do desafio 01 (inalterado no Ato I)
│   ├── Main.java
│   ├── abstracts/Item.java
│   └── ...
└── test/                              # NOVO — a sua rede
    ├── ItemTest.java
    ├── MusicalScoreTest.java
    ├── MovieTest.java
    ├── SerieTest.java
    └── CollectionStoreTest.java

```

No mundo Maven isso vira `src/main/java` e `src/test/java`, e é a estrutura que você vai encontrar em qualquer projeto Spring. Não vamos migrar agora — uma ferramenta nova por desafio. Mas repare que a *ideia* é a mesma: teste vive separado do código, e não vai para produção junto. Ao terminar, atualize `docs/ESTRUTURA.md` registrando a pasta `test/`: documentação que não acompanha a mudança vira mentira em três semanas.

3. Setup — JUnit sem build tool

Um jar, um classpath, zero XML. Baixe na raiz da pasta do desafio:

```

cd java/04-colecoes-streams/acervo-da-quarentena

curl -L -o junit-console.jar \
  https://repo1.maven.org/maven2/org/junit/platform/junit-platform-console-standalone/6.1.3/junit-platform-console-standalone-6.1.3.jar

```

Compilar código e testes juntos, e executar a suíte:

```

javac -encoding UTF-8 -cp junit-console.jar -d out $(find src test -name '*.java')

java -jar junit-console.jar execute --class-path out --scan-class-path

```

Sobre a versão — e sobre o que você vai ler na internet

A geração atual é o **JUnit 6** (baseline Java 17; você está no 21, tudo certo). Mas **99% dos tutoriais, cursos e respostas de fórum dizem "JUnit 5"** — e o código deles funciona igual no seu projeto, porque o pacote continua sendo `org.junit.jupiter.api` e as anotações são as mesmas. O que mudou entre o 5 e o 6 é baseline de JDK e entranhas da plataforma, não a API que você vai escrever.

Confira no Maven Central se saiu versão mais nova da linha 6.x e ajuste a URL. Saber descobrir a versão atual de uma dependência é parte do trabalho — e "o tutorial de 2021 mandou usar 5.7" é a origem de metade dos projetos desatualizados que você vai herdar.

Adicione `junit-console.jar` e `out/` ao `.gitignore`. Jar baixado não se versiona.

4. Ato I — Montar a rede

Regra de ouro do Ato I: você não altera nenhuma linha dentro de `src/`. Nem para consertar os tipos que você sabe que estão lá. Se um teste falhar porque o código está errado, *anote e siga* — o conserto é do Ato II. Se você não resistir e corrigir agora, perdeu a chance de sentir o que este desafio ensina.

4.1 O que a suíte precisa cobrir

Isto é o mínimo. Nomes de método e organização em arquivos são decisão sua — e você defende no review.

#	GRUPO	O QUE VERIFICAR	FERRAMENTA
1	Duração — Partitura	42 compassos a 60 bpm dão 2,8 min. Um caso não inteiro também (96 a 84 bpm).	<code>assertEquals</code> com delta
2	Duração — Filme	a duração informada é a duração devolvida	<code>assertEquals</code>
3	Duração — Série	temporadas [36, 29] a 25 min/ep dão 1625,0 min	<code>assertEquals</code>
4	Duração total	o acervo de exemplo soma 1753,7 min, sem arredondar antes de somar	<code>assertEquals</code> com delta
5	Construtor rejeita lixo	bpm 0; dificuldade 9; ano 2049; ano 1200; título vazio; título só com espaços; duração negativa; array de episódios vazio	<code>assertThrows</code>
6	Catalogar duplicado	mesmo código duas vezes lança a sua exceção — e o acervo continua com o item original	<code>assertThrows</code>
7	Buscar inexistente	o comportamento que você escolheu no desafio 01, agora fixado por teste	<code>assertThrows</code>
8	Cópia defensiva	<code>acervo.listAll().clear()</code> não esvazia o acervo. E o que mais essa chamada faz? Descubra e teste.	<code>assertThrows</code> + <code>assertEquals</code>
9	Contrato equals/hashCode	um Filme e uma Partitura com o mesmo código são iguais; códigos diferentes não são; objetos iguais têm o mesmo hashCode	<code>assertEquals</code> , <code>assertNotEquals</code>
10	Sessão da noite	item <code>ILEGIVEL</code> nunca entra; a ordem gulosa é a do enunciado 01; 120 min devolvem só o filme; 130 min devolvem três itens	<code>assertEquals</code> em lista
11	Sessão — bordas	minutos negativos lançam <code>IllegalArgumentException</code> ; zero minutos devolve lista vazia (e não estoura)	<code>assertThrows</code> , <code>assertTrue</code>
12	Acervo vazio	duração total 0; <code>listAll()</code> vazia; agrupamento vazio; sessão vazia. Nada de <code>NullPointerException</code> .	vários

Sobre o item 8 — tem coisa escondida ali

Você escreveu `Collections.unmodifiableList(new ArrayList<>(...))`. São *duas* proteções empilhadas, e elas falham de maneiras diferentes. Escreva o teste, veja o que acontece de verdade, e venha me dizer: uma das duas é redundante? O que `List.copyOf` faria diferente? Não pesquise a resposta antes de rodar — rode primeiro, tenha uma opinião, depois confira.

4.2 Como um teste JUnit se parece

Um exemplo curto, de um problema que **não é o seu**, só para você ver o formato. A tradução para o acervo é sua:

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class TermometroTest {

    @Test
    void converteZeroCelsiusParaTrintaEDoisFahrenheit() {
        Termometro t = new Termometro();           // Arrange
        double f = t.paraFahrenheit(0.0);          // Act
        assertEquals(32.0, f, 0.001);              // Assert
    }

    @Test
    void rejeitaTemperaturaAbaixoDoZeroAbsoluto() {
        Termometro t = new Termometro();
        assertThrows(IllegalArgumentException.class,
            () -> t.paraFahrenheit(-300.0));
    }
}
```

Três coisas para reparar: o nome do método *descreve o comportamento* e é lido como frase; o corpo tem três blocos (prepare, execute, verifique — chamam isso de **AAA**); e `assertEquals` de `double` leva um terceiro argumento. Por quê? Rode `System.out.println(0.1 + 0.2);` num `main` qualquer e você terá a resposta na tela.

5. Regras que separam o júnior do sênior

R1 — Um teste que não pode falhar não é um teste

Você já viveu isso: no desafio 01 a sua suíte imprimia "todos os testes passaram" e não tinha como reprovar nada. Depois de escrever cada bloco de testes, **quebre o código de propósito** — troque um `+` por um `-`, um `<=` por um `<`, um `60` por `61` — e confirme que algum teste fica vermelho. Depois desfaça. Isso chama-se **teste de mutação**, tem ferramenta de mercado (PIT) e foi exatamente o que eu fiz no seu código antes de aprovar o desafio 01. Entregue um `test/MUTACOES.md` curto: que mutação você fez, qual teste pegou.

R2 — Um teste, uma razão para falhar

Se o seu teste tem seis `assertEquals` sobre seis assuntos diferentes, quando ele falhar você vai saber que *algo* quebrou, não *o quê*. Testes pequenos custam mais linhas e economizam mais tempo. Qual é a exceção legítima a essa regra? Pense em quando várias verificações descrevem *um único* comportamento.

R3 — O nome do teste é a documentação

`testeDuracao1()` não conta nada. `itemIlegivelNaoEntraNaSessaoDaNoite()` conta. Quando alguém abrir a sua suíte daqui a seis meses — e esse alguém é você — a lista de nomes precisa ser lida como a especificação do acervo. Se o nome ficou grande demais para caber, o teste provavelmente está fazendo duas coisas (ver R2). Existe uma anotação do JUnit para nomes em português com espaços e acento: descubra qual, e decida se vale a pena usar ou se o nome do método já basta.

R4 — Teste comportamento, não implementação

Testar que a duração de uma partitura é 2,8 min é comportamento: continua valendo se você reescrever o cálculo. Testar que `getCompass()` devolve o que o construtor recebeu é implementação: o teste quebra quando você refatora, sem que nada tenha realmente quebrado. Suíte cheia de teste de getter é suíte que o time aprende a ignorar. **Não teste getters triviais.** E anote: quais dos 12 itens da tabela são comportamento, e algum é implementação disfarçada?

R5 — Teste não conversa com teste

Cada teste monta o que precisa e não depende de nenhum outro ter rodado antes. Se a sua suíte passa em uma ordem e falha em outra, você não tem uma suíte — tem um roteiro. O JUnit tem uma anotação que roda um preparo antes de *cada* teste: use, e entenda por que não é "antes de todos".

6. Ato II — Usar a rede

Agora sim. Com a suíte verde, você vai mexer no código do desafio 01 — e a regra muda:

Protocolo do Ato II

Uma mudança de cada vez. Rode a suíte. Verde? Próxima. Vermelho? Você acabou de descobrir algo — ou quebrou o código, ou o teste estava errado. **Descubra qual antes de continuar**, e nunca "conserte" apagando o teste.

#	FAXINA PENDENTE DO DESAFIO 01	O QUE VOCÊ VAI SENTIR
1	Tipos na API pública: <code>validadeTitle</code> , <code>outher</code> , <code>gratter</code> , <code>Compaas</code> , <code>DEMAGED</code> , <code>ILEGIBLE</code>	renomear coisa pública dá medo sem teste. Com teste, é entediante — e entediante é o objetivo.
2	Idioma consistente no projeto inteiro (<code>obterTotalItens</code> convive com <code>getTotalDuration</code>) — critério de aceite 13 do desafio 01, o único que ficou aberto	a suíte também está em algum idioma. Escolha antes de começar.
3	O <code>enum ItemType</code> existe mas só é usado como <code>ItemType.MOVIE.name()</code> . Faça-o valer como tipo.	aqui a suíte vai te cobrar de verdade. É a refatoração mais interessante da lista.
4	Cabeçalho: <code>=== Relatório principal DA QUARENTENA ===</code> — consulte a seção 4.2 do desafio 01	trinta segundos. Está aqui porque sobreviveu a quatro rodadas.
5	<code>dir.dir.desconhecido</code> na ficha do filme	o dado já traz o prefixo que o <code>describe()</code> também escreve. Onde é o lugar certo do prefixo?
6	Os 4 avisos de <code>javac -Xlint:all</code>	trate ou justifique cada um, por escrito. Um deles (<code>this-escape</code>) é sutil e vale a pesquisa.
7	<code>describe()</code> imprime <code>118.0</code> e a sessão imprime <code>118,0</code> , na mesma tela	decida de quem é a responsabilidade de formatar. Não existe resposta única — existe justificativa.
8	Justificar <i>checked</i> vs. <i>unchecked</i> das suas exceções, no <code>PLAN.md</code> , e deixar assinatura e implementação coerentes	pendência arrastada desde a primeira revisão. Agora você tem testes que <i>documentam</i> o comportamento — use-os no argumento.

7. Critérios de aceite

- ☐ A suíte roda com o comando da seção 3 e o console do JUnit reporta **0 failures**.
- ☐ Os 12 grupos da tabela 4.1 estão cobertos.
- ☐ Nenhum teste depende de outro teste ter rodado antes (R5).
- ☐ Nenhum `System.out.println` dentro dos testes. Quem reporta é o JUnit.
- ☐ Nenhum teste de getter trivial (R4).
- ☐ Existe `test/MUTACOES.md` com pelo menos **3** mutações feitas e qual teste pegou cada uma (R1).

- ☐ Ao final do Ato II, os 8 itens da faxina estão feitos **e a suíte continua verde**.
- ☐ `javac -Xlint:all` não emite aviso — ou cada aviso restante está justificado por escrito.
- ☐ O programa principal continua funcionando: `java Main` ainda imprime os números da seção 4.2 do desafio 01.
- ☐ `docs/ESTRUTURA.md` foi atualizado com a pasta `test/`.
- ☐ `.gitignore` cobre `junit-console.jar` e `out/`.
- ☐ Idioma consistente — **no código e nos testes**.

Bônus — depois que o principal estiver de pé

- Troque os testes repetitivos de duração por um `@ParameterizedTest` com `@CsvSource`. Compare com a versão anterior: ficou mais legível ou menos? Guarde as duas para o review.
- Use `assertAll` em um lugar onde ele faz sentido e explique por que ali, e não nos outros.
- `@Nested` para agrupar os testes de sessão da noite por cenário.
- Escreva um teste que *documenta um bug conhecido e ainda não corrigido*. O JUnit tem anotação para isso. Descubra qual — e por que ela é melhor do que comentar o teste.

O que não vale

- Fazer o Ato II antes do Ato I. É o coração do desafio.
- Alterar `src/` durante o Ato I — nem um typo.
- Apagar, comentar ou afrouxar um teste para ele passar. Se ele está vermelho, ele está te dizendo alguma coisa. *Você já fez a escolha certa nessa situação uma vez.*
- Pedir a suíte pronta ao Claude. Erro de compilação, "como funciona `assertThrows`", "o que é um lambda": pode e deve perguntar.
- Copiar teste de fórum que você não sabe explicar. No review, eu pergunto.

8. Por que este desafio existe

No desafio 01 você aprendeu a construir. Aqui você aprende a **mudar sem quebrar** — e a segunda habilidade vale mais que a primeira, porque código de verdade passa 10% da vida sendo escrito e 90% sendo alterado por gente que não estava lá quando ele nasceu.

Repare no que aconteceu com você nas rodadas 1.5 e 1.6: você mexeu no código para melhorar e introduziu uma regressão que zerou uma seção inteira do relatório — duas vezes seguidas, sem perceber. Não foi falta de cuidado. Foi falta de rede. Ninguém consegue segurar um programa inteiro na cabeça a cada mudança; a diferença entre quem quebra produção e quem não quebra raramente é atenção — é ter, ou não ter, quem avise.

No mercado isso é filtro de entrada. Take-home sem teste é reprovado antes de alguém olhar a sua arquitetura. Em entrevista de vaga sênior, "como você testaria isso?" vem logo depois de "como você implementaria isso?", e a segunda resposta pesa mais que a primeira. E quando você chegar ao Spring, vai reencontrar exatamente estas anotações — `@Test`, `@BeforeEach`, `assertThrows` — dentro de `@SpringBootTest` e `@DataJpaTest`. A API que você aprende aqui é a mesma que você vai usar lá.

Teste não existe para provar que o seu código funciona. Existe para você poder mudá-lo amanhã sem medo. Medo é o que faz programador bom escrever código ruim.

Mestre Yoda · Desafio 02 · A rede não impede a queda. Ela deixa você tentar o salto.